Attributes-Effect-on-OOD-Performance

Matthew Liu and Luca Williams

Overview

This project examines how prompt attributes affect a generative model's performance on out-of-distribution tasks, which include both never-seen concepts and rare configurations of familiar objects encountered during training. We use membership inference to determine whether a task is out of distribution and apply VQA score to evaluate how closely generated images align with ground-truth descriptions. Using one generative model and three prompt settings consisting of base mid-OOD, low-OOD, and high-OOD tasks, we identify clear differences in model behavior. For mid-OOD tasks, additional visual attributes and descriptive modifiers tend to reduce performance, while increased prompt length often improves alignment. In low-OOD tasks, prompt attributes show little consistent influence on performance. For high-OOD tasks, the model struggles across all prompt designs, indicating limits in its ability to generalize to highly unfamiliar scenarios.

Replication Instructions

In the following instructions, we use the wine prompt as an example to illustrate each step of the pipeline. All required files have already been created and saved in the artifact and can be used for verification.

1. **Determine an OOD task with membership inference**

   To assess whether a prompt is out of distribution (OOD), we run `membership_inference/Membership_Inference_script.py` five times, each time using a different seed image (indexed from full_0001.png to full_0005.png). The script is executed using the following command:

   ```
   ./membership_inference/Membership_Inference_script.py \
   --seed-image ./membership_inference/images/wine/full_0001.png \
   --prompt "a glass bottle of wine filled to the brim"
   ```

   For each run, the script outputs a feature vector consisting of the minimum LPIPS distance at each noise strength, along with a binary classification indicating whether the image is likely in-training or out-of-training. We record the feature vector from each run and compute the average of the final element (corresponding to the highest noise strength) across the five vectors.

   If this average exceeds 0.25, the prompt is classified as likely out of distribution. Larger average values indicate a greater degree of out-of-distribution behavior, reflecting the model's reduced ability to reconstruct the seed image under high noise.

2. **Ground truth testing with VQAScore**

   After we have established a task is OOD, we want to ensure that the VQAScore method is able to reliably distinguish between images that successful accomplish the task, and images that fail at the task.

   For this, we use the VQAScore in the `t2v_metrics` folder. `t2v_metrics` is a submodule of another repository and specific directions can be found in the README.md here. Make sure to cd into `VQAScore` to run it. The simple directions for setup are as follows:

```
git clone https://github.com/linzhiqiu/t2v_metrics
cd t2v_metrics

conda create -n t2v python=3.10 -y
conda activate t2v
conda install pip -y

conda install ffmpeg -c conda-forge
pip install -e . # local pip install
```

   To set up the `t2v_metrics` submodule, we used the Colgate Supercomputer to be able to load and use open weight models and the VQAScore in general. Required libaries for the repo are located in `./t2v_metrics/pyproject.toml`. We want to note that although we ended up using OpenAI's gpt-4o model for the VQAScore, the requirements for this folder still require extensive computational resources, and it should not be anticipated to be run locally without reconcilation of conflicting requirements. Instructions to use other models other than gpt-4o are in the original repo as well.

   Once the VQAScore repo is set up, we took 5 images that aligned with the OOD task we were trying to elicit, and 5 images that did not align. To ensure that the VQA method would work for a given task, we want to see that it can discern between the true and false images. For example, we would want to see a picture of a truly full wine glass get scored near a 1.0 while a normal glass of wine was scored much lower. See examples in any of the `ground_truths` folders for reference. The VQAScore can be run with the folllowing command after images and base prompts are set up:

```
python ground_truth.py \
  --api_key YOUR_OPENAI_API_KEY
```

3. **Prompt stratification**

   Run `./prompting/generate_prompts.py` in the prompting folder to generate the task_prompts.csv file. This file contains prompts with systematically varied combinations of prompt attributes.

Use `./prompting/base_prompt.txt` and `./prompting/system_prompt.txt` to store the base and system prompt for the prompt generation. Examples can be found in a `prompts.txt` file within a task folder in the `./generated_images` directory.

Inputs for `generate_prompts.py` include the following:

- `--prompt_file`: str type, file path for the resulting csv
- `--system_prompt`: str type, file path for the system_prompt.txt file
- `--base_prompt`: str type, file path for the base_prompt.txt file
- `--descriptor_words`: int type, number of max descriptor words to stratify a prompt across
- `--visual_attributes`: int type, number of max visual attribute words to stratify a prompt across
- `--iterations`: int type, amount of itterations to stratify a prompt for on a single set of combinations

Example script call:

```
python generate_prompts.py \
  --api_key YOUR_OPENAI_API_KEY \
  --prompt_file wine_prompts.csv \
  --descriptor_words 4 \
  --visual_attributes 3
```

4. **Image generation**

   Run `image_generators/ImageGeneration.py` with the stratified prompt CSV specified via the command line to generate images for each prompt. The generated images are automatically saved to the designated output directory (e.g., generated_images/, with wine-specific outputs stored under generated_images/wine/). This step may require sufficient billing allowance to complete successfully.

   ```
   python ImageGeneration.py \
   --csv_file ./prompting/prompts/wine_prompts.csv \
   --output_dir generated_images/wine \
   --prefix wine
   ```

5. **Obtain VQA score**

   Similar to the use of VQA with ground_truths, the VQAScore will be obtained from the `t2v_metrics` submodule. This time, we will upload the generated images to a specified folder on the JupyterHub IDE along with the generated prompt csv. We will use the `vqa.py` script to parse the csv, evaluate a given image, and update the csv with the score from that image belonging to the corresponding prompt. Inputs: - `--input_file`: str type, prompt file containing variables and filename information - `--image_dir`: str type, directory path to folder with images for evaluation - `--base_prompt`: str type, base prompt to evaluate images against

- `--api_key`: str type, api_key for OpenAI API

Example:

```
python vqa.py \
  --input_file wine_prompts.csv \
  --image_dir ./images/wine/ \
  --base_prompt "a wine glass completely full to the brim with wine" \
  --api_key YOUR_OPENAI_API_KEY
```

6. **Statistical Analysis**

After VQAScore's have been added to the CSV, we used the `main.R` script to regress variables onto the resulting VQAScore. This can be done in VSCode with the right extentions or RStudio. It takes the path to the csv as an input and outputs a regression table like this:

```
Coefficients:
                      Estimate Std. Error t value Pr(>|t|)
(Intercept)            0.142319   0.073082   1.947  0.05267 .
word_count             0.015470   0.003307   4.677 4.89e-06 ***
descriptor_words      -0.051464   0.015961  -3.224  0.00144 **
num_visual_attributes -0.125707   0.029516  -4.259 2.97e-05 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.2761 on 236 degrees of freedom
Multiple R-squared:  0.1231,    Adjusted R-squared:  0.1119
F-statistic: 11.04 on 3 and 236 DF,  p-value: 8.249e-07
```

Future Directions

An important direction for future work is to address the mismatch between the target model used for membership inference and the model used for image generation during prompting. Identifying or developing a unified model that supports both text-to-image and image-to-image pipelines would enable more consistent evaluation and reduce uncertainty introduced by cross-model assumptions. Future work should also expand the scale of the pipeline by increasing the number of prompts and generated images, which is currently limited by computational and fiscal resources. Greater scale would allow for stronger statistical confidence, clearer correlations between specific prompt attributes and model performance, and more reliable comparisons across different types of OOD tasks. Testing VQAScore on tasks with a wider variety of models would also benefit the pipeline by allowing us to find the strongest way to measure each task as the measures tends to struggle on some (rubik cub with missing corner) as compared to others (wine completely full). With sufficient expansion, this framework could also be used to compare how different VQA-based evaluation models behave across tasks, offering deeper insight into how evaluation methodology itself influences

performance assessment under unfamiliar conditions.

Contributions

Matthew

- Membership inference recreation + testing: 6-8 hrs
- Image generation script + creating images: 6-8 hrs
- Poster intro/methodology/conclusion: 2 hrs

Luca

- VQAScore adaptation + testing: 8 hrs
- Prompt stratification methods: 6 hrs
- Poster background/intro/further directions/results: 2 hrs

Both

- Prompt idea brainstorm: 1 hr
- Literature review: 2-3 hrs each
- First milestone proposal draft: included in work for membership inference and VQAScore
- Final artfact readme writeup: 2-3 hrs each

Repository Guide

- `./analysis`
  - Folder containing R script to regress prompt variables against VQAScore outcome.
- `./documents`
  - Directory for miscellaneous documents from brainstorming, proposals, and milestones.
- `./generated_images`
  - Directory containing a folder for each task tested
  - Each task contains:
    - `ground_truths`: ground truth materials containing 5 true images and 5 false images as well as VQA outputs on a base prompt
    - `images`: generated images from the stratified prompt csv
    - `analysis.txt`: results from regression with R
    - `prompts.txt`: system prompts and base prompts used to stratify prompts
    - `./..._prompts.csv`: csv containing prompts, variables, and results for a given task
- `./image_generators`

  - contains `./ImageGeneration.py` which is the main script for generating images taken from prompts in a csv

- `./membership_inference`
  - Directory with materials for membership inferences script in python and notebook form.
  - Also contains `images` which is a folder with images to test membership inference on for a given task, and results from that test

- `./prompting`
  - Contains `generate_prompts.py` which is the main script for stratifying on prompts.
  - `base_prompt.txt` and `system_prompt.txt` are both txt files containing the system prompt and base prompt for the prompt generation, examples can be found in the generated images folder under a `prompt.txt` file for a given task.

- `./VQAScore`
  - contains scripts for VQAScore and submodule folder of the repository for the VQAScore (`./t2v_metrics`)

References

GenAI-Bench: Evaluating and Improving Compositional Text-to-Visual Generation

GenAI Confessions: Black-box Membership Inference for Generative Image Models